6CM504 Concurrency and communication

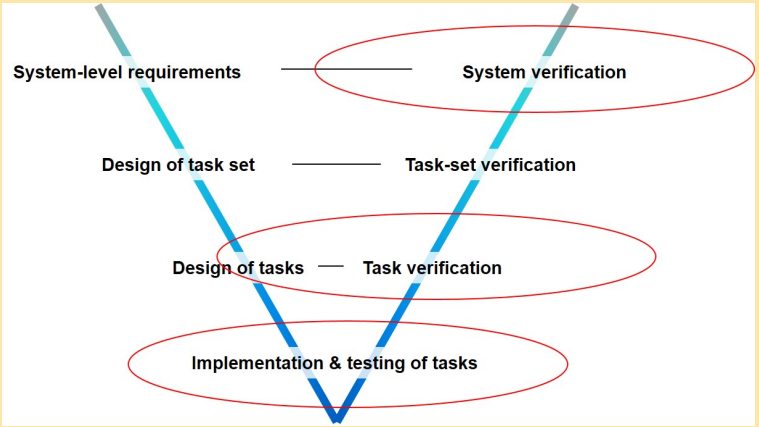
Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

This week



This week, we will continue to use all that we have learned about CSP—modelling, algebra, semantics, refinement, and concurrency—and apply it to a further typical embedded systems case study that allows us to reason about correctness at a level relevant to task verification.

Table of Contents

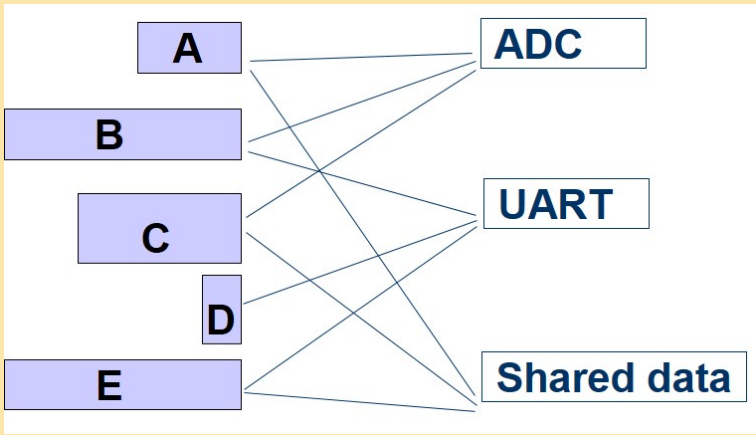
Mutual exclusion

1. Consider two (source code) processes (or tasks!) `P1` and `P2` whose access to a shared resource takes place only within a critical region

```
P1=                              P2 =
BEGIN                            BEGIN
  <critical region 1>              <critical region 2>
END;                             END;
```

2. Safety: the most important safety property of a piece of code like this is that no more than one of these processes may be inside its critical region at any one time
3. Liveness: however there is a key liveness property—the algorithm should not introduce deadlocks. If a process halts outside of it critical region, this should not prevent other processes from proceeding

What about these shared resources?



Even our microcontroller environment is a whole melting pot of shared resources—the integration and management of these resources is down to the software we write.

Table of Contents

A first attempt

1. We can try to write some source code that achieves this

```
P1 =
BEGIN
  flag1:= true
  WHILE flag2 DO nothing
  <critical region 1>
  flag1:= false
END
```

```
P2 =
BEGIN
  flag2:= true
  WHILE flag1 DO nothing
  <critical region 2>
  flag2:= false
END
```

Will this work?

1. Assume the flags represent global shared variables

```
P1 =                              P2 =
  BEGIN                             BEGIN
  flag1:= true                      flag2:= true
  WHILE flag2 DO nothing            WHILE flag1 DO nothing
  <critical region 1>               <critical region 2>
  flag1:= false                     flag2:= false
  END                               END
```

2. In fact this code *may* work in a single threaded, non pre-emptive environment but will fail in any other
3. Can we investigate this with a CSP model?

Model components

1. Processes `Process1` and `Process2`
2. Variables `flag1` and `flag2`

```
enter.1, leave.1, enter.2, leave.2

writeflag.1.1.T, writeflag.1.2.T,
    writeflag.1.1.F, writeflag.1.2.F
writeflag.2.1.T, writeflag.2.2.T,
    writeflag.2.1.F, writeflag.2.2.F
readflag.1.1.T, readflag.1.2.T,
    readflag.1.1.F, readflag.1.2.F
readflag.2.1.T, readflag.2.2.T,
    readflag.2.1.F, readflag.2.2.F
```

More succinctly, channels (1)

1. Some useful identifiers

```
-- Declare process identifiers
Id = {1, 2}
-- Declare a bool type for clarity
Bool = {T, F}
```

2. Declare sets of channels

```
-- Events identifying critical sections
channel enter, leave: Id
CRITS = {| enter, leave|}
```

3. Access events identifying process, flag, and channel

```
-- Flag access events
channel writeflag, readflag  : Id . Id . Bool
FLAGS = {| writeflag, readflag |}
```

Representing variables as processes

1. We can represent flags as processes

```
-- A flag can be represented as a process
-- It can be read from, or written to
Flag(i, b) =
  readflag ?j!i!b -> Flag(i, b)
  []
  writeflag?j!i?c -> Flag(i, c)
```

2. And create them

```
-- Create two different flags F1 and F2
-- with identifiers and initial values
F1 = Flag(1, F)
F2 = Flag(2, F)
```

Representing code as processes (1)

1. We can model statements in our source code using the corresponding channels we have declared
2. Assignment:
   - In `P1`, the assignment statement `Flag1:= true` may be represented by `writeflag.1.1.T -> SKIP`
3. While loop:
   - In `P1`, the while loop `WHILE flag2 DO nothing` may be represented by `readflag.1.2.false -> SKIP`

Representing code as processes (2)

1. We can now create a CSP process that describes the behaviour of the source code we saw earlier

```
Process(i, j) =
  writeflag!i!i!T ->
    readflag !i!j!F ->
      enter.i ->
        leave.i ->
          writeflag!i!i!F -> Process(i, j)


Process1 = Process(1, 2)
Process2 = Process(2, 1)
```

Bringing it together

1. The final system is the two interleaved processes
2. And the the two interleaved flags
3. Synchronising on the events used to access flags
   - Note the processes never communicate between themselves, hence we interleave them
   - Nor do the flags communicate between themselves—these are just models of variables—so we interleave them also

```
-- The system is the interleaved processes
-- in parallel with interleaved flags

System =
  (Process1 ||| Process2)
  [|{| FLAGS |}|]
  (Flag1 ||| Flag2)
```

The final system

1. Now we have a full model of the system
   - The source code manipulating the variables
   - The variables themselves
   - The critical regions
2. Question: Does our algorithm, and our source code, achieve what we want it to achieve?

Table of Contents

Dekker's algorithm

1. Dekker's algorithm is the first known correct solution to the mutual exclusion problem
   - Invented by the Dutch mathematician Edsger Dijkstra
   - It allows two thread to share a single resource without conflict, using only shared memory for communication
2. The algorithm is widely used because it does not rely on atomic reads and writes
   - Therefore highly portable between architectures and languages
   - Disadvantages include being limited to two processes, and reliant on busy waiting

Dekker's algorithm: the basics

1. The algorithm uses two flags `flag1` and `flag2` (both of which are boolean), and a record of whose turn it is `turn` (which is a process ID)
2. Both of the flags are initialised to false (to indicate that neither participant is in a critical section), and the turn to 1 (to indicate that process 1 may go first)

Dekker's algorithm: the pseudo code (1)

```
P1 =
  flag1:= true
  IF turn = 1 THEN
    WHILE flag2 DO nothing
  ELSE
  BEGIN
    flag1:= false
    WHILE turn = 2 DO nothing
    flag1:= true
    WHILE flag2 DO nothing
  END;
  <critical region 1>
  turn:= 2
  flag1:= false
```

Dekker's algorithm: the pseudo code (2)

```
P1 =
   flag2:= true
   IF turn = 2 THEN
     WHILE flag1 DO nothing
   ELSE
   BEGIN
     flag2:= false
     WHILE turn = 1 DO nothing
     flag2:= true
     WHILE flag1 DO nothing
   END;
   <critical region 2>
   turn:= 1
   flag2:= false
```

Dekker's algorithm: properties of interest

1. The properties we are most interested in of this algorithm are that:
   - Only one process can enter a critical section at any one time
   - A process is not presented from leaving the critical section by the lock
2. We can capture this using two safety specifications
   - We assume our model uses the events `enter.i` and `leave.i` to model entering and leaving the critical section

Table of Contents

Summary: assignment 2

1. The mutual exclusion examples we have studied today will form the basis of assignment 2
2. Details of the assignment will be given in the lab. In summary you will be asked to:
   - Implement the mutual exclusion example we have studied, in FDR
   - Verify, or otherwise, the behaviour of this model
   - Implement a model of Dekker's algorithm, building on this
   - Verify, or otherwise, the behaviour of this model
3. Submission details will be given out in the lab
4. This assignment is worth 50% of your module mark
5. Strict rules regarding plagiarism will be enforced

Parting remarks → SKIP

"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult."

C. A. R. Hoare

"Beware of bugs in the above code; I have only proved it correct, not tried it."

Donald Knuth